

COMP0008 Computer Architecture and Concurrency

Lecture 3: MIPS Processor (Part 1)

Kevin Bryson

K.Bryson@ucl.ac.uk

Background Reading:

Patterson & Hennessey, Chapter 2.

But it may be worth reading this after we have covered up to the topic of high-level languages ... to synthesize and consolidate the different topics since they explain all aspects in combination (with quite a lot of assumed background knowledge).

Overview of next few weeks ...

We are going to build up our understanding of the MIPS32 processor architecture from ...

- The Instruction Set Architecture (ISA) given in terms of hardware instructions and numeric register references.
- Moving onto assembly language aspects that include the addition of pseudo-instructions, names registers for particular purposes and symbolic labels.
- Then onto how high-level languages are compiled into assembly language and use of the hardware stack for function calling (this is when Chapter 2 could be useful).
- The gcc toolchain for MIPS – compiler / assembly / linker and loader – and some real MIPS assembly code.

Overview of this Lecture

- The previous lecture examined a number of key concepts about fixed size binary numbers (signed/unsigned, big/little-endian, sign/zero-extension, etc.).
- These concepts will be important when discussing different aspects of the MIPS32 Instruction Set Architecture (ISA).
- In this lecture we go into more detail about:
 - what instructions are present within the MIPS32 ISA
 - how these instructions are encoded in terms of “machine code” within memory
 - what they do in terms of semantics (their meaning).

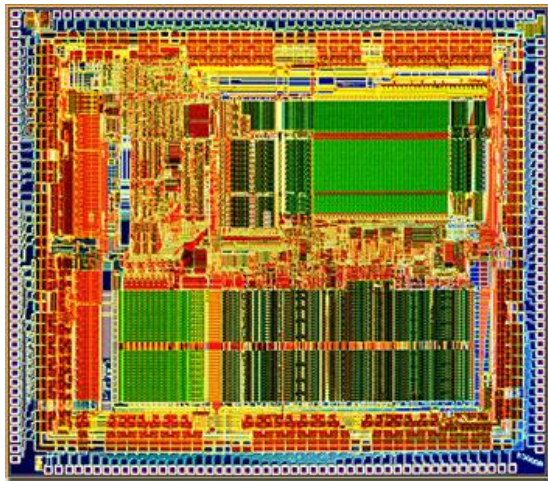
R3000 MIPS Processor

Released: 1988

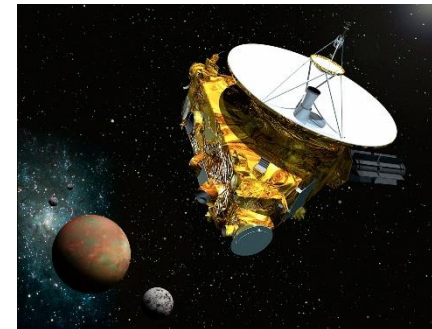
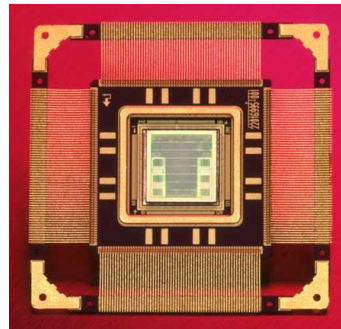
Transistor count: 115,000

Used in high-end workstations (e.g. Silicon Graphics SGI Personal IRIS 4D/20)

This processor was used to render 3D scenes in 90s movies like Jurassic Park or Terminator 2 and more recently a radiation hardened version of the R3000 flew to Pluto.



Silicon chip view of the R3000 processor



Radiation hardened Mongoose-V version of the R3000 processor used in the New Horizon space probe to Pluto.

Mikroelectronics “Clicker” microcontroller - like an Arduino but based on a PIC32 chip (MIPS32 architecture)

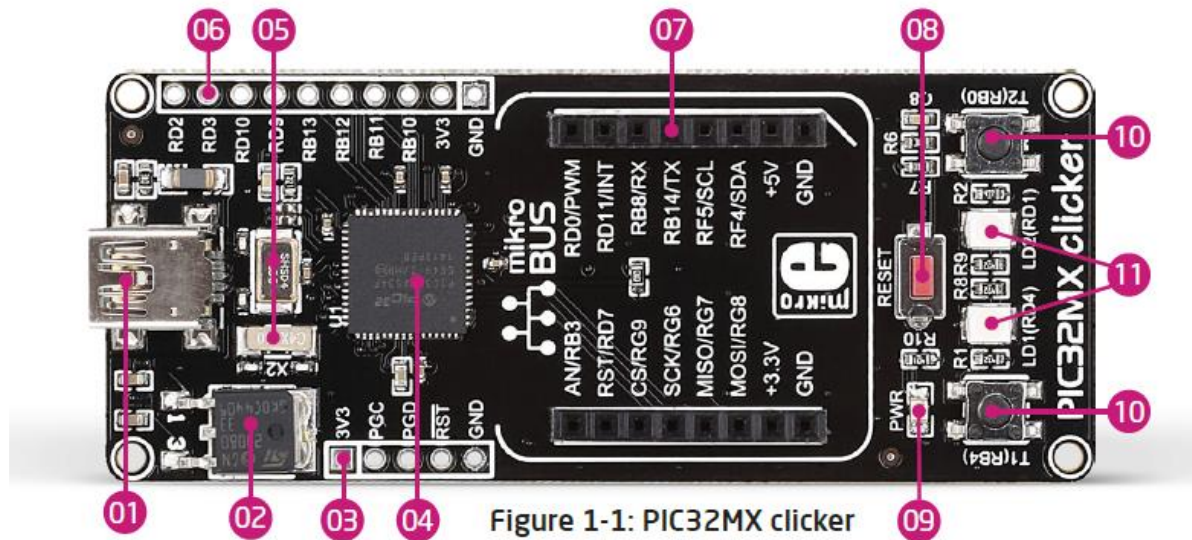
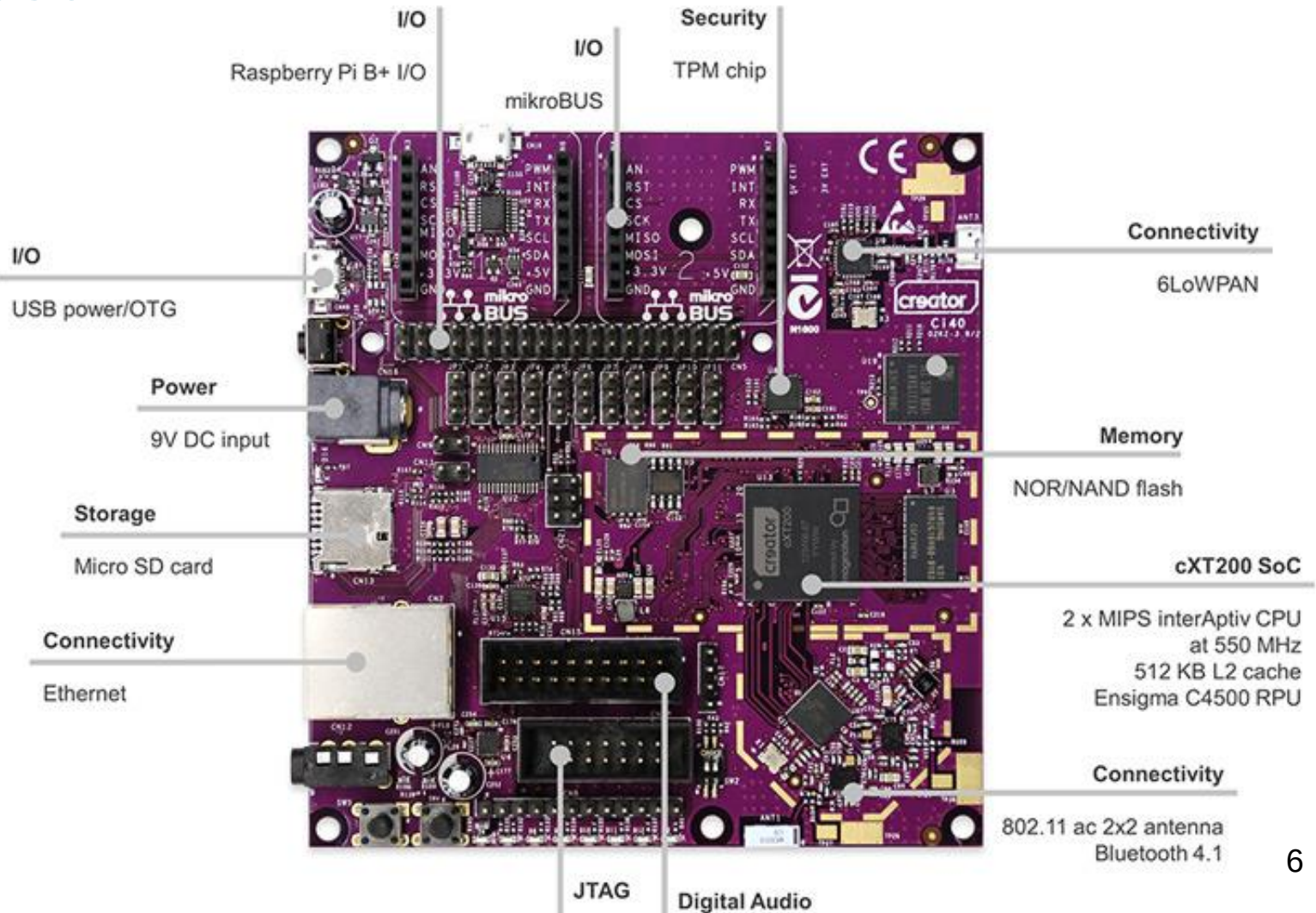


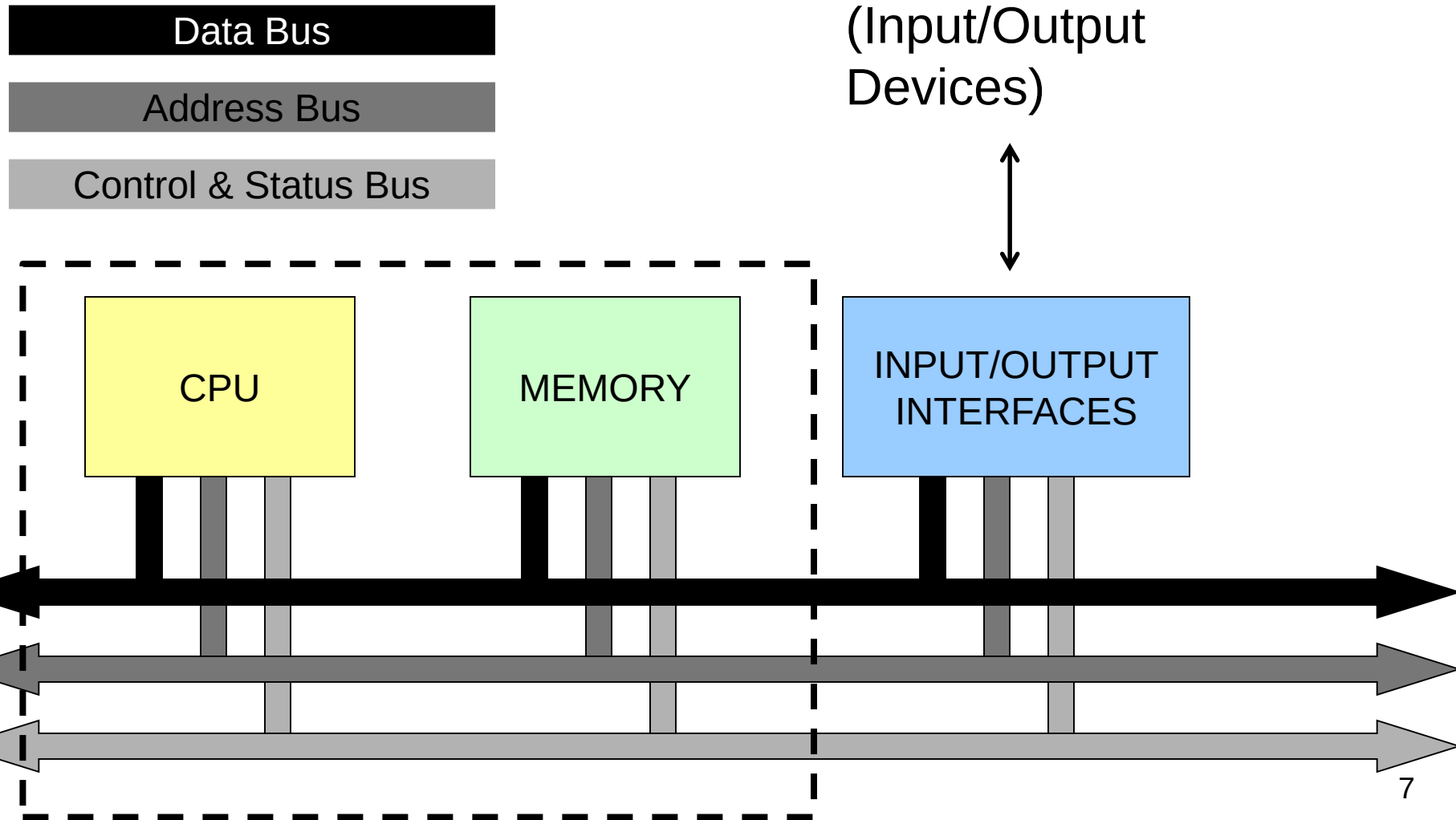
Figure 1-1: PIC32MX clicker

Microcontrollers generally have sensors and “effectors” and run hardware such as washing machines ...

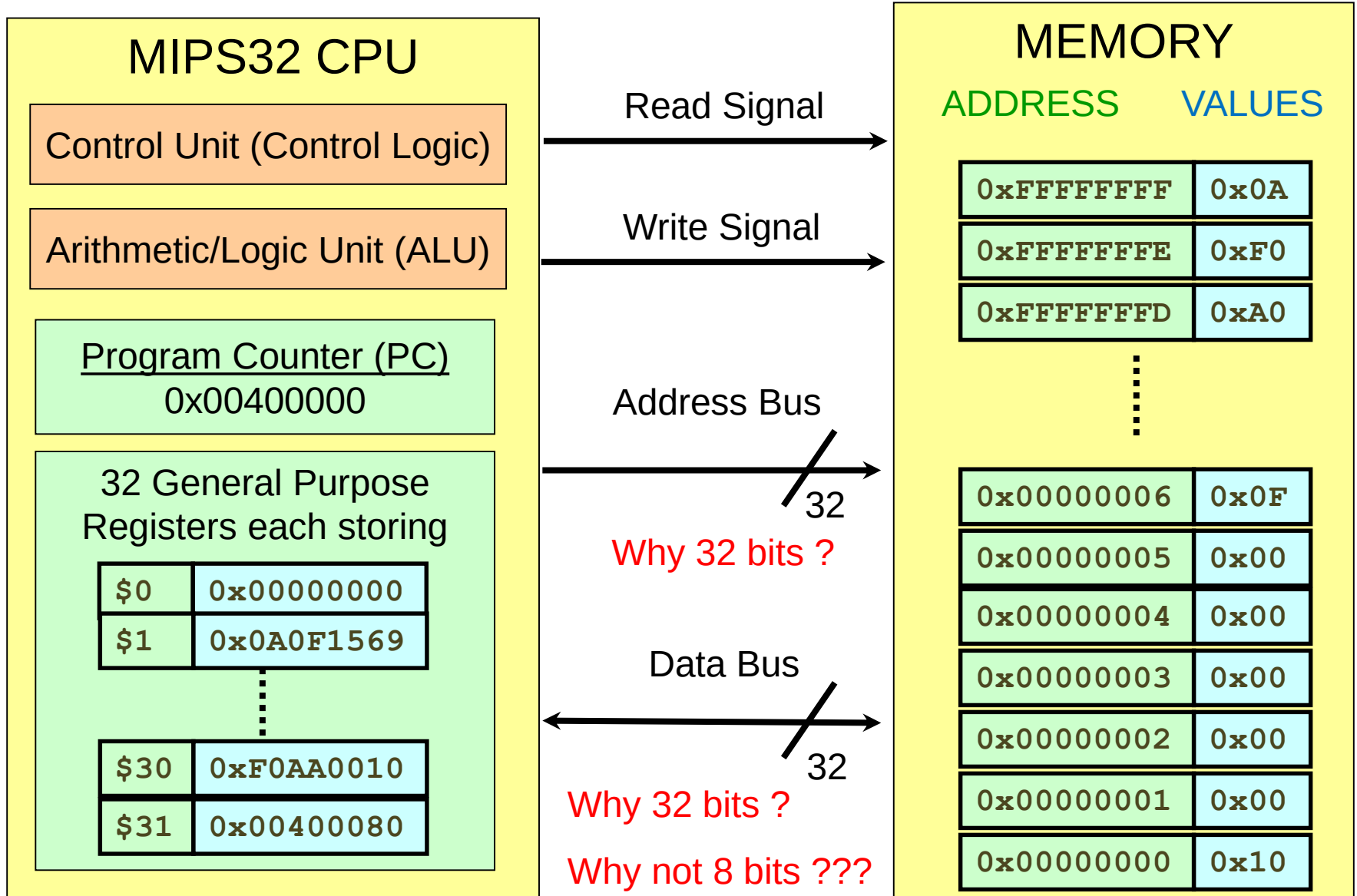
The Imaginations Creative Ci40 single board computer - like a Raspberry Pi but with a MIPS32 processor



A Processors' (or Programmers') Abstract View of a Computer

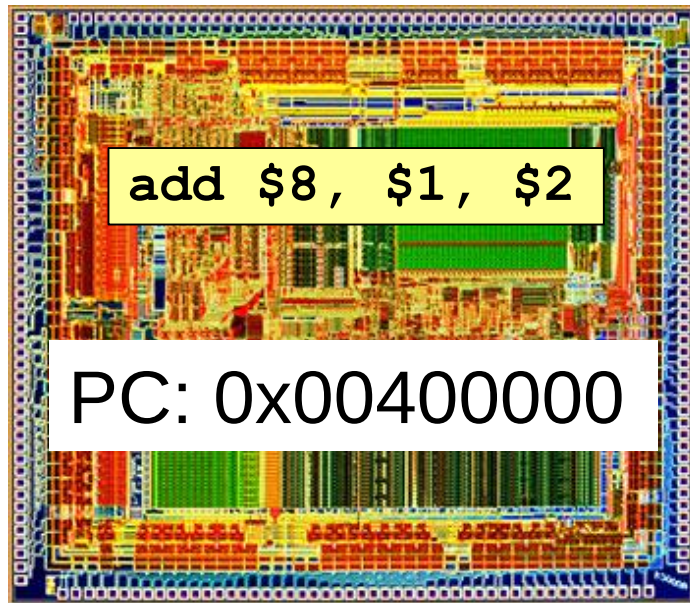


MIPS32 Processor $\leftrightarrow$ Memory



Fetch / Execute Cycle

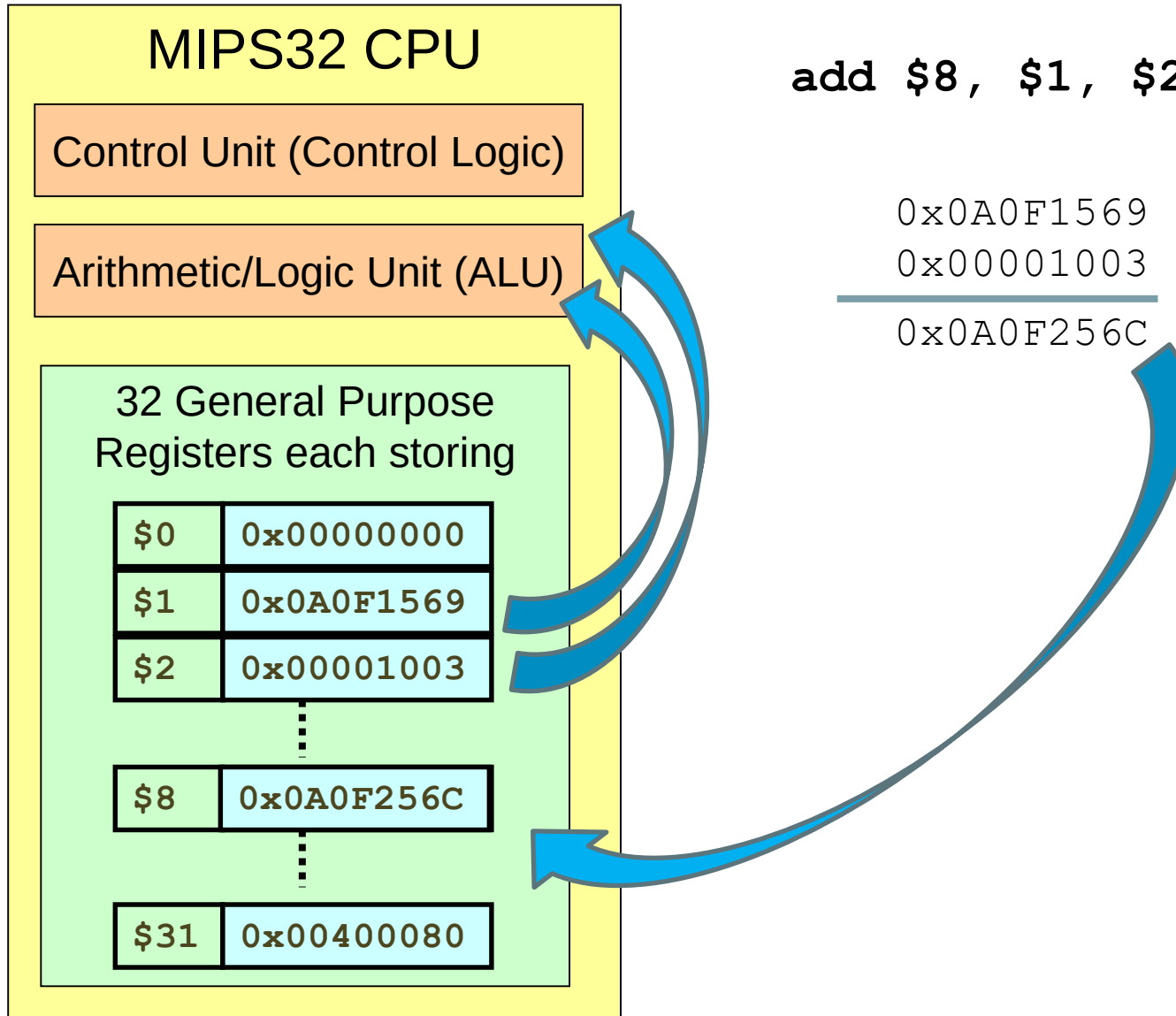
1. Processor puts PC values onto address bus.
2. Processor triggers 'read signal'
3. Memory returns 32-bit instruction at this address (4 consecutive bytes)
4. Processor executes this instruction.
5. Increment PC to PC + 4 (moving the PC to the next instruction)



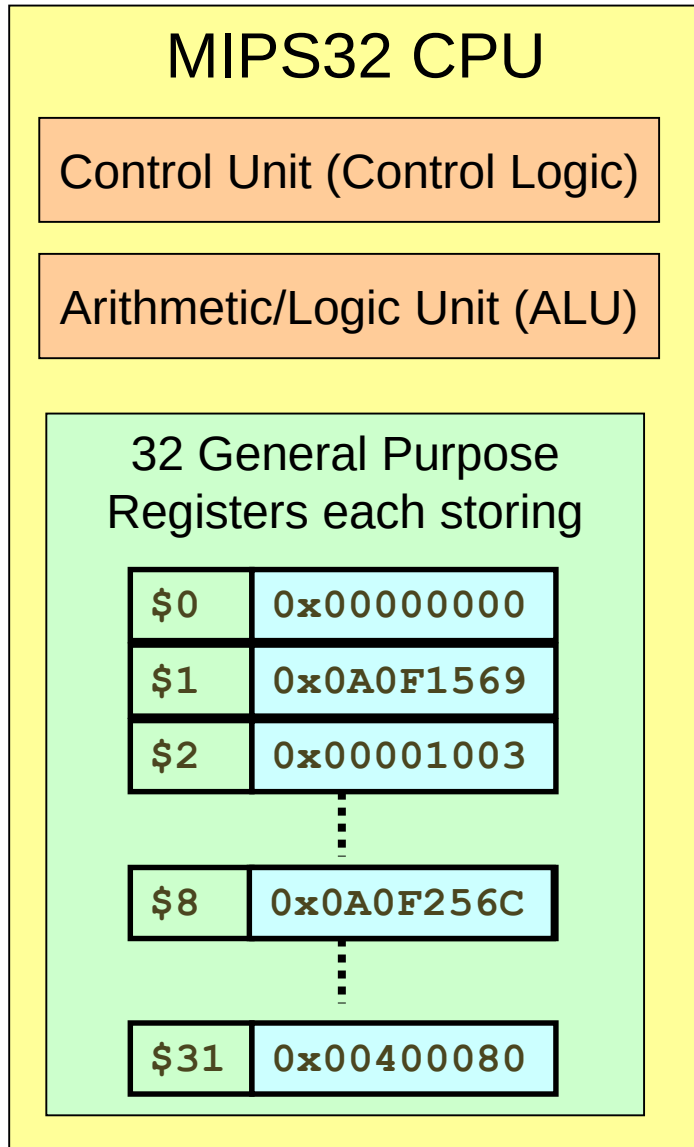
0x00224020

MEMORY	
ADDRESS	VALUES
0xFFFFFFFF	0x0A
⋮	
0x00400005	0x0F
0x00400004	0x00
0x00400003	0x20
0x00400002	0x40
0x00400001	0x22
0x00400000	0x00
⋮	
0x00000000	0x10

MIPS32 Executing Instructions



But how do we operate on data in memory?



- RISC based “load and store” architecture (only load and store instructions access memory – **all other instructions access registers only**)
- So adding two values stored in memory together and putting the answer back into memory requires 4 RISC instructions:
 - Load value 1 from memory into register.
 - Load value 2 from memory into register.
 - Add together register values and put answer into another register.
 - Finally store result back into memory.